UNIT – V: AGILE SOFTWARE DESIGN AND DEVELOPMENT

1. Analyze the importance of Refactoring and Continuous Integration in maintaining Agile software quality.

Refactoring

Refactoring is the process of improving the internal structure of existing code without changing its external or observable behaviour. It improves readability, organization and design.

Importance of Refactoring:

1. **Removes code smells** such as duplicated code, long methods and large classes.
2. **Reduces technical debt** accumulated due to quick design decisions.
3. **Makes code easier to maintain** and understand.
4. **Supports changing requirements** because Agile software evolves every sprint.
5. **Improves design** by applying principles such as SOLID.
6. **Makes future changes safer** and easier.

Continuous Integration

Continuous Integration (CI) is the practice of frequently integrating code changes into a shared main branch, where each integration is automatically built and tested.

Importance of CI:

1. Detects integration problems early.
2. Finds broken builds and regressions quickly.
3. Provides fast feedback to developers.
4. Avoids large and difficult **"merge day"** problems.
5. Keeps the codebase in a continuously releasable state.
6. Supports frequent and low-risk Agile delivery.

Relationship between Refactoring and CI

Refactoring changes the internal structure of working code. Automated regression tests in CI verify that the behaviour has not changed. Therefore, **refactoring keeps the code clean, while CI continuously verifies its quality**.

Conclusion: Refactoring and CI together help Agile teams maintain clean, reliable and continuously releasable software.

---

2. Identify any four common Refactoring Techniques.

Refactoring techniques improve the internal structure of code without changing its external behaviour. The following are common techniques given in the PPT.

1. Extract Method

A long method is divided into smaller, well-named methods, with each method performing one clear task.

Example:
A `printInvoice()` method can be divided into:

- `calculateTotal()`
- `printTotal()`
- `printThankYouMessage()`

This helps solve the **Long Method** code smell.

2. Rename Method / Variable

A method, variable or class is given a clearer and more meaningful name.

Example:

```
int d;
```

can become

```
int elapsedTimeInDays;
```

This makes the purpose of the variable easier to understand.

3. Extract Class

A large class with multiple responsibilities is divided into smaller, focused classes.

For example, an `Employee` class handling both personal information and salary calculation can be divided into separate classes such as `ContactInfo` and `PayrollCalculator`.

4. Replace Conditional with Polymorphism

A large `if/else` or `switch` statement based on object type is replaced with subclasses that provide their own behaviour.

For example, different employee types such as `Engineer`, `Manager` and `Intern` can each implement their own `calculatePay()` method.

Other techniques:

- Inline Method
- Move Method
- Introduce Parameter Object
- Replace Magic Number with Constant
- Decompose Conditional.

Conclusion: Refactoring techniques make code simpler, cleaner and easier to maintain while keeping its behaviour unchanged.

---

3. Describe the Open-Closed Principle (OCP) with a suitable example.

The **Open-Closed Principle (OCP)** states that software entities such as classes, modules and functions should be **open for extension but closed for modification**.

In simple words, we should be able to add new functionality **without modifying existing, already-tested code**.

Why OCP is important:

1. Reduces the risk of breaking existing functionality.
2. Makes new features easier to add.
3. Reduces regression problems.
4. Supports Agile development where new features are added frequently.
5. Encourages the use of interfaces, abstraction and polymorphism.

Example:

Suppose an `AreaCalculator` calculates the area of different shapes.

Without OCP:

The class contains:

- If shape is Circle → calculate circle area
- Else if shape is Rectangle → calculate rectangle area
- Else if shape is Triangle → calculate triangle area

Whenever a new shape is added, the existing method must be modified.

With OCP:

Create a common `Shape` interface:

- `Circle implements Shape`
- `Rectangle implements Shape`
- `Triangle implements Shape`

Each shape provides its own `area()` method.

Now, when a new shape is required, we can create a new class without changing the existing `AreaCalculator`.

Benefits:

- Existing code remains unchanged.
- New functionality can be added easily.
- Testing becomes safer.
- Regression risk is reduced.

Conclusion: OCP is especially useful in Agile because it allows new features to be added with minimum changes to existing code.

---

4. Explain the role of Git in Agile Software Development.

Git is a distributed version control system. It records changes to files and allows developers to track history, compare versions, revert mistakes and work together safely.

Role of Git in Agile:

1. **Tracks changes**
   - Every change to the code can be recorded.
2. **Supports collaboration**
   - Multiple developers can work on the same project without overwriting each other's work.
3. **Branches**
   - Developers can create separate branches for individual features or user stories.
4. **Merging**
   - Completed feature branches can be merged back into the main codebase.
5. **Pull Requests**
   - Changes can be reviewed by teammates before being merged.
6. **Rollback**
   - Previous versions can be recovered if a mistake is introduced.
7. **Supports Continuous Integration**
   - Git works together with CI pipelines that build and test code after changes are pushed.

Feature-Branch Workflow

In Agile, a common approach is to create a **short-lived feature branch** for each new feature or user story. After development, review and successful CI testing, the branch is merged into the main branch.

Best practices:

- Make small, meaningful commits.
- Use clear commit messages.
- Keep feature branches short-lived.
- Use pull requests for code review.
- Do not merge broken code into the main branch.

Conclusion: Git supports Agile collaboration, safe development, code review and frequent integration.

---

5. Why are Design Principles important when requirements change frequently?

Agile requirements change frequently because new features are added and existing requirements are modified every sprint. Therefore, the software design must remain flexible and easy to change.

Importance of Design Principles:

1. **Keep code flexible**
   - Principles help developers create code that can adapt to new requirements.
2. **Reduce complexity**
   - They prevent the codebase from becoming difficult to understand.
3. **Reduce technical debt**
   - Good design avoids accumulating poor design decisions.
4. **Support maintainability**
   - Well-designed code is easier to modify and test.
5. **Reduce regression risk**
   - Changes in one part are less likely to break unrelated parts.
6. **Support refactoring**
   - Principles guide developers while improving existing code.
7. **Improve extensibility**
   - Principles such as OCP allow new behaviour to be added without modifying existing code.

SOLID Principles

The main design principles used in the PPT are **SOLID**:

- **S** – Single Responsibility Principle
- **O** – Open/Closed Principle
- **L** – Liskov Substitution Principle
- **I** – Interface Segregation Principle
- **D** – Dependency Inversion Principle.

Conclusion: Design principles provide common rules for keeping an evolving Agile codebase flexible, understandable, maintainable and safe to change.

---

6. What is Continuous Integration (CI)? Mention its major benefits.

Continuous Integration (CI) is a software development practice in which developers frequently merge their code changes into a shared main branch. Each integration is automatically **built and tested**.

CI Pipeline:

Code Commit → Build → Automated Tests → Static Analysis → Package → Staging → Release

1. **Code Commit** – Developer pushes changes.
2. **Build** – CI server builds the application.
3. **Automated Tests** – Unit, integration and acceptance tests are executed.
4. **Static Analysis** – Code quality and possible problems are checked.
5. **Package** – A deployable artifact is created.
6. **Deploy to Staging** – The artifact is deployed for further validation.
7. **Release** – Approved software can be released.

Major Benefits:

1. Detects integration problems early.
2. Finds defects and regressions quickly.
3. Provides fast feedback to developers.
4. Reduces integration conflicts.
5. Keeps the software continuously releasable.
6. Supports frequent and low-risk delivery.
7. Makes software quality easier to monitor.

CI Best Practices:

- Commit small and frequent changes.
- Keep the build fast.
- Fix broken builds immediately.
- Maintain a reliable automated test suite.

Conclusion: CI is an important Agile practice that continuously checks code quality and detects problems early.

---

7. What is the role of Git in Agile Software Development?

Note: This question is the same as Question 4 in the given important-question list. Therefore, the **same answer can be written for both questions**.

Git is a distributed version control system that records changes, maintains history and supports collaboration between developers.

Main roles of Git:

- Tracks code changes.
- Maintains project history.
- Supports feature branches.
- Allows merging of changes.
- Supports pull requests and peer review.
- Helps developers revert mistakes.
- Works together with Continuous Integration.
- Supports parallel development by Agile teams.

A common Agile workflow is:

Create Feature Branch → Develop → Commit → Push → Pull Request → Review → CI Check → Merge

Git therefore provides the version-control foundation required for collaborative Agile development.

---

8. How does Version Control support team collaboration in Agile projects?

A **Version Control System (VCS)** records and manages changes to project files over time. It allows developers to track history, compare versions, revert mistakes and work in parallel.

Ways Version Control supports collaboration:

1. **Parallel Development**
   - Different developers can work on different features at the same time.
2. **Branches**
   - Each feature or user story can be developed in its own branch.
3. **Commit History**
   - Teams can see what changed, when it changed and why.
4. **Merge**
   - Completed work from different branches can be combined.
5. **Pull Requests**
   - Team members can review code before it is merged.
6. **Conflict Management**
   - If two developers change the same part of a file, the conflict can be identified and resolved.
7. **Safe Recovery**
   - Previous versions can be restored if a mistake occurs.
8. **Supports CI**

- Version control provides the changes that trigger automated build and test pipelines.

Agile Feature-Branch Workflow:

Feature/User Story → Short-lived Branch → Development → Pull Request → Code Review → CI → Merge

Short-lived branches and frequent merging reduce large merge conflicts and support Agile's frequent integration approach.

Conclusion: Version control provides a safe and organized way for Agile teams to collaborate, review, integrate and manage changing code.